Enhanced Task Manager CLI

โครงงานปลายภาค · Final Project — สัปดาห์ที่ 12

รายวิชา: Object-Oriented Programming (การเขียนโปรแกรมเชิงวัตถุ)

นายอเสช ปัญญาวงค์ · รหัสนักศึกษา 663380247-9

```
> python main.py
```

Final Presentation · 5–7 นาที

แนะนำโปรเจกต์ (Introduction)

1 · Introduction

โปรแกรมบริหารจัดการงานส่วนตัวบนหน้าจอคำสั่ง (Command-Line Interface) — รวมงานไว้ที่เดียว จัดหมวดด้วยป้ายกำกับ ค้นหาและกรองได้ทันที

งาน 3 ประเภท

· งานทั่วไป (Task) — รหัส, รายละเอียด, สถานะ

· งานมีกำหนดส่ง (DueDateTask) — วันครบกำหนด

· งานตามความสำคัญ (PriorityTask) — High / Medium / Low

ฟีเจอร์หลัก

· ค้นหา (Search) ทั้งชื่องานและป้ายกำกับ

· กรองหลายเงื่อนไขพร้อมกัน (Filter)

· เรียงลำดับได้ 5 แบบ (Sort)

· บันทึกอัตโนมัติลงไฟล์ JSON (Persistence)

เป้าหมายการเรียนรู้

· การสืบทอดคลาส (Inheritance)

· พฤติกรรมต่างตามชนิด (Polymorphism)

· แยกส่วนตามหน้าที่ (Modularity)

· จัดการข้อผิดพลาด (Error Handling)

ปัญหาที่โปรแกรมแก้ไข (Problem Solved)

2 · Problem Solved

งานตกหล่น

ปัญหา

งานหลายวิชากระจายคนละที่ — จดสมุดบ้าง จดโทรศัพท์บ้าง ⟶ ลืมกำหนดส่ง

ทางแก้

รวมงานทุกอย่างไว้ที่เดียว พร้อมวันครบกำหนด และบันทึกอัตโนมัติ — ปิดโปรแกรมแล้วข้อมูลไม่หาย

แยกงานด่วนไม่ชัด

ปัญหา

ไม่รู้ว่าควรทำอะไรก่อน — งานด่วนปนอยู่กับงานทั่วไป

ทางแก้

กำหนดระดับความสำคัญ (High / Medium / Low) และเรียงลำดับงานตามความสำคัญหรือวันส่งได้ทันที

หางานเก่าไม่เจอ

ปัญหา

รายการงานยาวขึ้นเรื่อย ๆ — ค้นหาด้วยตาอย่างเดียวทำได้ช้า

ทางแก้

ติดป้ายกำกับ (Tags) ให้แต่ละงาน แล้วใช้ค้นหา (Search) / กรอง (Filter) หลายเงื่อนไขพร้อมกัน

เป้าหมาย: ลดงานตกหล่น และหางานเจอได้ในไม่กี่วินาที

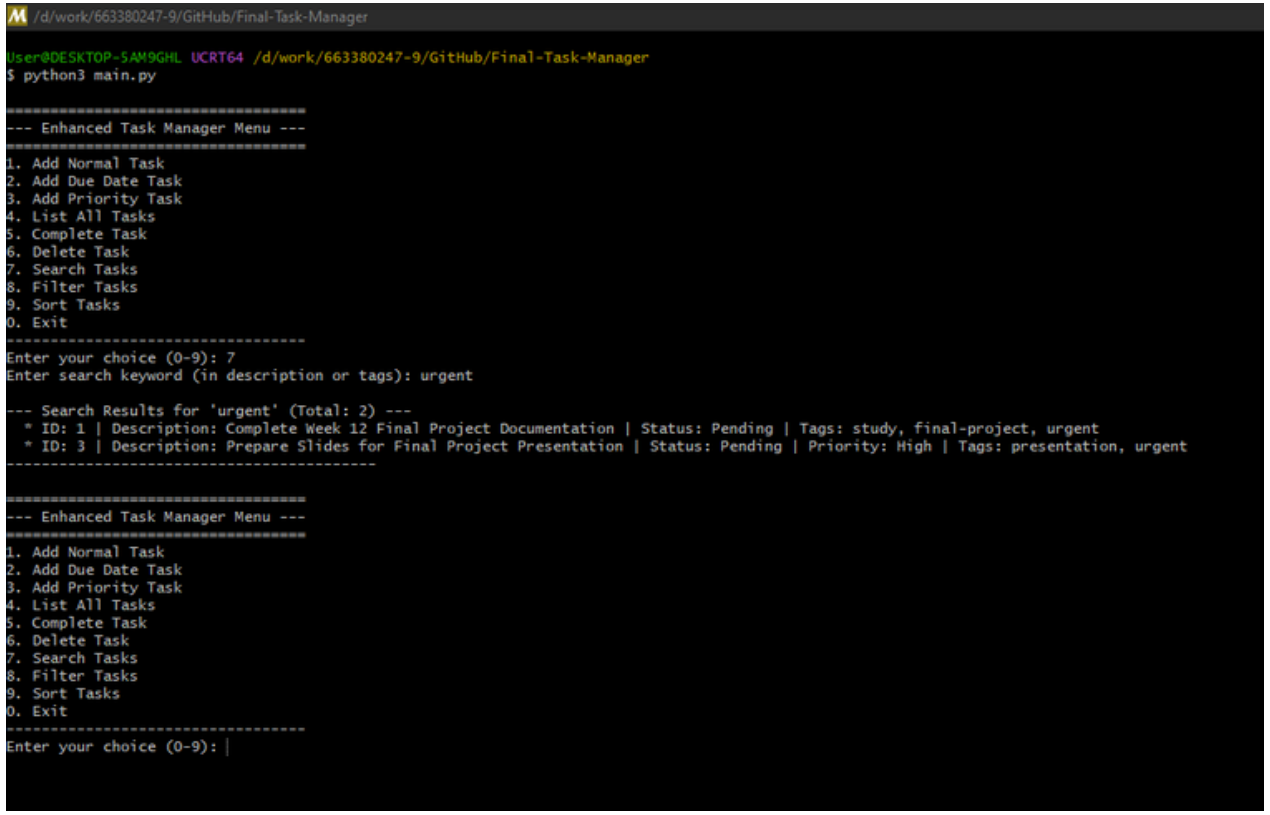
สาธิตการใช้งานจริง (Live Demo)

3 · Live Demo

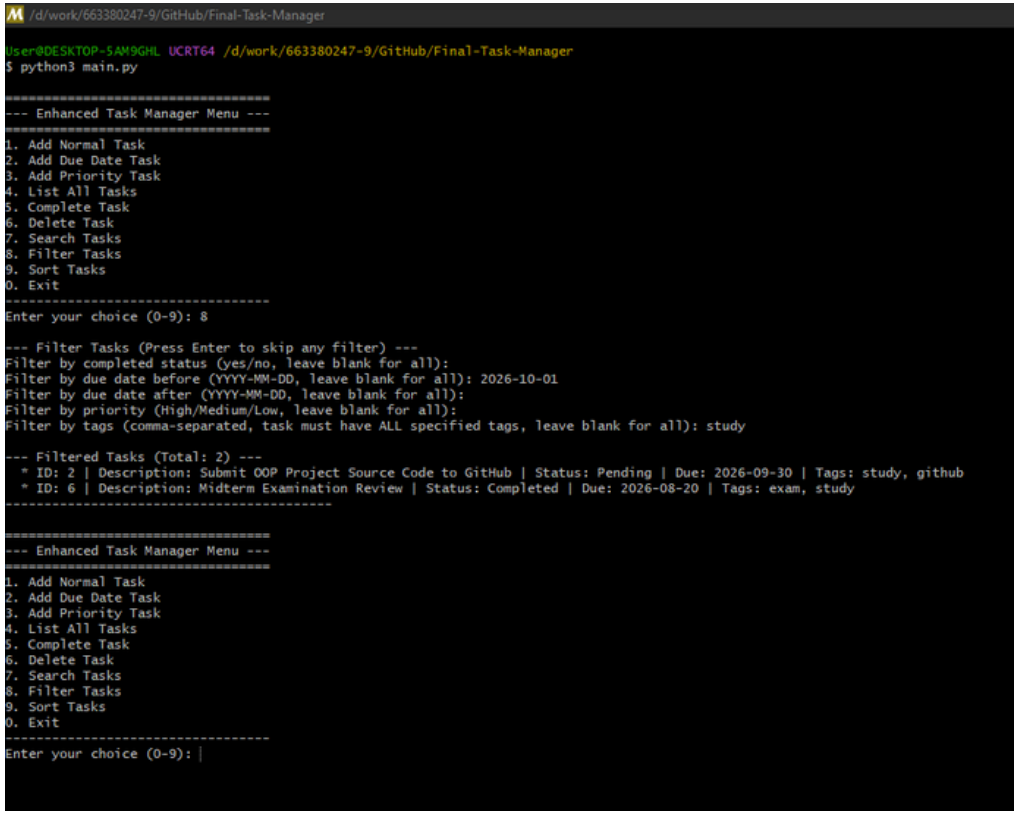
ลำดับการสาธิต 6 ขั้นตอน — ใช้ข้อมูลตัวอย่างใน data/tasks.json (ไม่ต้องใช้อินเทอร์เน็ต)

1. แสดงรายการงานทั้งหมด (List All) — ตัวอย่าง 6 งาน ครบ 3 ประเภท พร้อมป้ายกำกับ
2. เพิ่มงานใหม่ (Add) — เลือกประเภทงาน กรอกชื่องาน วันส่ง/ความสำคัญ และป้ายกำกับ
3. ค้นหา (Search) — พิมพ์คำว่า “urgent” ⟶ พบทั้งงานที่มีคำในชื่อและในแท็ก (ไม่สนตัวพิมพ์เล็ก-ใหญ่)
4. กรอง (Filter) — รวมหลายเงื่อนไขพร้อมกัน เช่น สถานะ + ช่วงวันส่ง + ป้ายกำกับ
5. เรียงลำดับ (Sort) — ตามความสำคัญ High ⟶ Medium ⟶ Low หรือตามวันส่ง / รหัสงาน
6. ปิดและลบงาน (Complete · Delete) — อัปเดตสถานะสำเร็จ หรือลบงานตามรหัส (ID)

ผลลัพธ์จริงจากการรันโปรแกรม



เมนู 7 · ค้นหา “urgent” — พบ 2 งาน (ตรงจากชื่อและแท็ก)



เมนู 8 · กรอง: กำหนดส่งไม่เกิน 2026-10-01 + แท็ก study — พบ 2 งาน

ภาพบันทึกจากการรันจริงของโปรแกรม · ภาพเพิ่มเติมอยู่ในโฟลเดอร์ screenshots/ ของโปรเจกต์

โครงสร้างโปรแกรม (Software Architecture)

4 · Architecture

โครงสร้างคลาส — OOP

Task — คลาสแม่ (Base Class)

id · description · completed · tags (ป้ายกำกับ)
add_tag / remove_tag / mark_complete

DueDateTask
(+ due_date วันครบกำหนด)

PriorityTask
(+ priority ความสำคัญ)

การสืบทอดคลาส (Inheritance): คลาสลูกได้คุณสมบัติจาก Task ทั้งหมด

พฤติกรรมต่างตามชนิด (Polymorphism): แต่ละคลาสมี to_dict() / __str__() ของตัวเอง

การแยกส่วนตามหน้าที่ (Separation of Concerns)

CLIInterface
ชั้นติดต่อผู้ใช้ — เมนู, รับคำสั่ง, แสดงผล

TaskManager
ตรรกะหลัก — เพิ่ม/ลบ/ค้นหา/กรอง/เรียงลำดับ

DataPersistence
บันทึกและโหลดข้อมูล JSON

data/tasks.json
ไฟล์เก็บข้อมูลถาวร

ทุกชั้นใช้โมเดลข้อมูลร่วมกันจาก task.py

ความท้าทายทางเทคนิคและวิธีแก้ (Challenges)

5 · Challenges

เรียงลำดับงานต่างชนิด (Polymorphic Sort)

ปัญหา: งานแต่ละชนิดมีข้อมูลไม่เท่ากัน — บางงานไม่มีวันส่ง บางงานไม่มีความสำคัญ ⟶ เรียงตรง ๆ จะพัง

ทางแก้: ใช้คีย์เรียง (Sort Key) แบบคู่: งานไม่มีวันส่งใช้ค่า date.max / ไม่มีความสำคัญใช้ค่า 99 ดันลงท้ายสุด + รหัสงาน (ID) เป็นคีย์รองให้ลำดับนิ่งเสมอ

กรองหลายเงื่อนไขพร้อมกัน (Multi-criteria Filter)

ปัญหา: ต้องรวมสถานะ + ช่วงวันส่ง + ความสำคัญ + แท็ก พร้อมกัน โดยไม่ให้โปรแกรมล่มเมื่อเจองานต่างชนิด

ทางแก้: กรองแบบต่อเนื่องทีละเงื่อนไข (Sequential Pipeline) + ตรวจชนิดงานก่อนเทียบวัน (isinstance) — ทำงานกับงานที่มีข้อมูลครบเท่านั้น

รองรับไฟล์ข้อมูลรุ่นเก่า (Backward Compatibility)

ปัญหา: ไฟล์งานเดิมที่บันทึกไว้ก่อนมีระบบแท็ก — ไม่มีฟิลด์ tags เลย

ทางแก้: เติมค่าเริ่มต้นอัตโนมัติ setdefault('tags', []) ตอนโหลด + ระบุชนิดงานด้วยฟิลด์ _type — เปิดไฟล์เก่าได้ไม่พัง

บทเรียนที่ได้จากโปรเจกต์ (Lessons Learned)

6 · Lessons Learned

การแยกโมดูล (Modularity)

แยกชั้นติดต่อผู้ใช้ / ตรรกะ / เก็บข้อมูล ออกจากกัน ⟶ ระบุตำแหน่งบั๊กได้ทันทีว่าอยู่ชั้นไหน และแก้ได้โดยไม่กระทบส่วนอื่น

การทดสอบอัตโนมัติ (Unit Tests)

ชุดทดสอบ 8 รายการ ครอบคลุมโมเดล ตรรกะ และการบันทึกข้อมูล ⟶ มั่นใจก่อนปรับโค้ดทุกครั้ง + จับการเชื่อมโยงข้ามไฟล์ได้เร็ว

ออกแบบเพื่อรองรับการเปลี่ยนแปลง

ฟิลด์ระบุชนิดงาน (_type) + ค่าเริ่มต้น (setdefault) ทำให้ระบบอ่านไฟล์ข้อมูลรุ่นก่อนหน้าได้อย่างราบรื่น

แผนการพัฒนาต่อยอด (Future Work) & Q&A

7 · Future Work & Q&A

หน้าจอกราฟิก (GUI)

เพิ่มหน้าจอแบบกราฟิกด้วย Tkinter / PyQt ให้ใช้งานง่ายขึ้น

ฐานข้อมูลจริง (Database)

ย้ายจากไฟล์ JSON ไปใช้ SQLite / PostgreSQL ที่รองรับข้อมูลจำนวนมาก

ระบบแจ้งเตือน (Notification)

เตือนงานใกล้ถึงกำหนดส่งผ่านระบบปฏิบัติการ หรือ LINE Notify

ผู้ใช้หลายคน (User Accounts)

ระบบเข้าสู่ระบบ แยกงานของแต่ละคนออกจากกัน

REST API

เชื่อมต่อผ่าน FastAPI เพื่อต่อยอดเป็นเว็บแอปหรือแอปมือถือ

ต่อยอดตามแนวทางใน PLAN.md — เอกสารการออกแบบและแผนงานของโปรเจกต์

ขอบคุณครับ

Questions & Answers — ยินดีตอบทุกคำถาม

Enhanced Task Manager CLI · นายอเสข ปัญญาวงค์ · 663380247-9